

Technische Hochschule Ingolstadt
Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

Camera Calibration

Oct 08, 2025

Welcome to the lecture!

Prof. Dr.-Ing. Björn Giesler

Technische Hochschule Ingolstadt / Fraunhofer IVI

Autonomous, Cooperating Systems

Email: bjoern.giesler@thi.de

- **Please register for the lecture on Moodle!**
  - Link: https://moodle.thi.de/course/view.php?id=10580
  - Key: **SensNT+DF#WS25**
- **News on Moodle:**
  - Slides for the lecture (upcoming lectures marked as WIP and are subject to change!)
  - Formulary, Exercises (Solutions provided after discussion in the lecture)
  - Exercises are optional, one task / sheet per week, but are highly recommended.
    They can be handed in for correction.
  - I will *try* to publish exercises before Wednesday‘s lecture so that initial questions can be discussed on Thursday.
- **Exam: Written exam, 90 minutes**
- **Consultation hours: Please arrange via e-mail to bjoern.giesler@thi.de.**
- **Please also don‘t hesitate to send questions, lecture feedback, change requests, etc.!**

Goal: Obtain 3D objects from video



- **Sensing with Cameras**
  - Camera Calibration continued
  - Object Detection using Viola-Jones
- **Neural Networks Intro / Recap**

A Quick Look At The Programming Exercise

Camera Calibration Continued

- **Cameras project 3D points to 2D according to** $\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{z} \underbrace{\begin{pmatrix} f & 0 & -c_u \\ 0 & af & -c_v \\ 0 & 0 & 1 \end{pmatrix}}_{\mathbf{K}} \begin{pmatrix} x \\ y \\ z \end{pmatrix}$ **, where**
  - $(x, y, z)^T$ is the coordinate of the 3D point,
  - $(u, v, 1)^T$ is the homogeneous coordinate of the projection in the image plane,
  - f is the focal distance of the camera, a is a scaling factor to compensate for non-square pixels, and
  - (c_u, c_v) is the image center (i.e. the point where the ray from the camera zero is normal on the image plane).
- **Together with a homogeneous transformation matrix** $\begin{pmatrix} & \mathbf{R} & & \mathbf{t} \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$

  the full projection equation becomes $\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} \mathbf{KR} & \mathbf{Kt} \end{pmatrix} \begin{pmatrix} x/z \\ y/z \\ 1 \end{pmatrix}$ **.**

  Let's give $\begin{pmatrix} \mathbf{KR} & \mathbf{Kt} \end{pmatrix}$ **the name** $\mathbf{P}$**. It's a 3x4 matrix of 12 unknowns.**

- **We can obtain $\mathbf{P}$ by solving the overdetermined system of linear equations**

$$\begin{pmatrix} u_n \\ v_n \\ 1 \end{pmatrix} = \mathbf{P} \begin{pmatrix} x_n \\ y_n \\ z_n \\ 1 \end{pmatrix}$$

with n >> 6 point correspondences, because each point gives *two* observation equations:

$$u = \frac{p_{11}x + p_{12}y + p_{13}z + p_{14}}{p_{31}x + p_{32}y + p_{33}z + p_{34}}$$
$$v = \frac{p_{21}x + p_{22}y + p_{23}z + p_{24}}{p_{31}x + p_{32}y + p_{33}z + p_{34}}$$

- **To make it simpler, we rewrite the equation as**

$$\begin{pmatrix} u'_n \\ v'_n \\ w_n \end{pmatrix} = \begin{pmatrix} \overbrace{p_{11} \quad p_{12} \quad p_{13} \quad p_{14}}^{\mathbf{A}^\top} \\ p_{21} \quad p_{22} \quad p_{23} \quad p_{24} \\ \underbrace{p_{31} \quad p_{32} \quad p_{33} \quad p_{34}}_{\mathbf{C}^\top} \end{pmatrix} \begin{pmatrix} x_n \\ y_n \\ z_n \\ 1 \end{pmatrix} = \begin{pmatrix} \mathbf{A}^\top \\ \mathbf{B}^\top \\ \mathbf{C}^\top \end{pmatrix} \begin{pmatrix} x_n \\ y_n \\ z_n \\ 1 \end{pmatrix}$$

$\mathbf{B}^\top$

with

$$\begin{pmatrix} u_n \\ v_n \\ 1 \end{pmatrix} = \begin{pmatrix} u'_n/w_n \\ v'_n/w_n \\ 1 \end{pmatrix}.$$

- **Rearranging…**

$$u_n = \frac{\mathbf{A}^\top \mathbf{X_n}}{\mathbf{C}^\top \mathbf{X_n}} \Rightarrow u_n \mathbf{C}^\top \mathbf{X_n} - \mathbf{A}^\top \mathbf{X_n} = 0$$

$$v_n = \frac{\mathbf{B}^\top \mathbf{X_n}}{\mathbf{C}^\top \mathbf{X_n}} \Rightarrow u_n \mathbf{C}^\top \mathbf{X_n} - \mathbf{B}^\top \mathbf{X_n} = 0$$

with $\mathbf{X_n} = \begin{pmatrix} x_n \\ y_n \\ z_n \\ 1 \end{pmatrix}$

gives us a system of linear equations

$$-\mathbf{X_n}^\top \mathbf{A} + u_n \mathbf{X_n}^\top \mathbf{C} = 0$$
$$-\mathbf{X_n}^\top \mathbf{B} + v_n \mathbf{X_n}^\top \mathbf{C} = 0$$

(we can transpose at will here because these multiply out as scalars)

which we can write as

$$\mathbf{M} \begin{pmatrix} \mathbf{A} \\ \mathbf{B} \\ \mathbf{C} \end{pmatrix} = \begin{matrix} -x_n & -y_n & -z_n & -1 & 0 & 0 & 0 & 0 & u_n x_n & u_n y_n & u_n z_n & u_n \\ 0 & 0 & 0 & 0 & -x_n & -y_n & -z_n & -1 & v_n x_n & v_n y_n & v_n z_n & v_n \end{matrix} \begin{matrix} \\ \\ \\ \\ \\ \\ \\ \\ \\ \\ \\ \\ \end{matrix} \begin{matrix} \mathrm{A} \\ \\ \\ \\ \mathrm{B} \\ \\ \\ \\ \mathrm{C} \\ \\ \\ \end{matrix} \begin{pmatrix} p_{11} \\ p_{12} \\ p_{13} \\ p_{14} \\ p_{21} \\ p_{22} \\ p_{23} \\ p_{24} \\ p_{31} \\ p_{32} \\ p_{33} \\ p_{34} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

- In the last lecture, we have seen that we can solve overdetermined systems of equations $\mathbf{Ar} = \mathbf{q} + \mathbf{e}$

  by introducing a function $S(\mathbf{r}) = \sum e_i^2$ and minimizing it by deriving and setting to zero.

  The result is $\mathbf{r} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{q}$.

- Here we have a system of the form $\mathbf{Ar} = 0$

  which can be solved by enforcing a Lagrange constraint for $\mathbf{r}^\top \mathbf{r} = 1$ of the form

  $$\mathcal{L}(\mathbf{r}, \lambda) = S(\mathbf{r}) + \lambda(\mathbf{r}^\top \mathbf{r} - 1)$$

  which, after derivation, leads to $\mathbf{A}^\top \mathbf{Ar} + \lambda \mathbf{r} = 0$ (please refer to the last lecture)

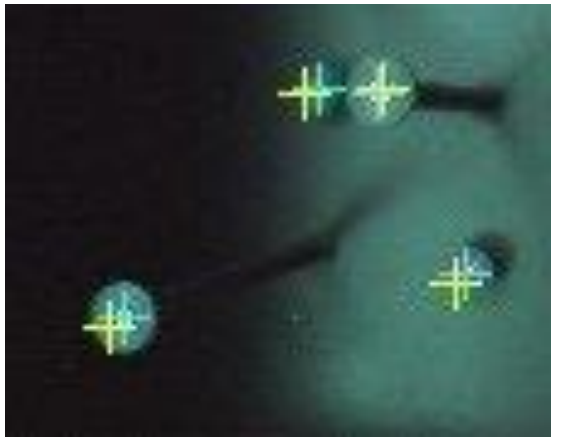
  $$\mathbf{A}^\top \mathbf{Ar} = -\lambda \mathbf{r}$$

  We can solve this Eigenvalue problem with the Singular Value Decomposition.

Camera Calibration Recap

- **We measure the remaining error for the calibration by *backprojecting* the 3D points into the image and determining the backprojection error:**

$$\begin{pmatrix} u_i' \\ v_i \\ 1 \end{pmatrix} = \mathbf{P} \begin{pmatrix} x_i \\ y_i \\ z_i \\ 1 \end{pmatrix} \qquad e = \sum \sqrt{u_i^{\,2} + u'^{\,2}_i} + \sum \sqrt{v_i^{\,2} + v'^{\,2}_i}$$



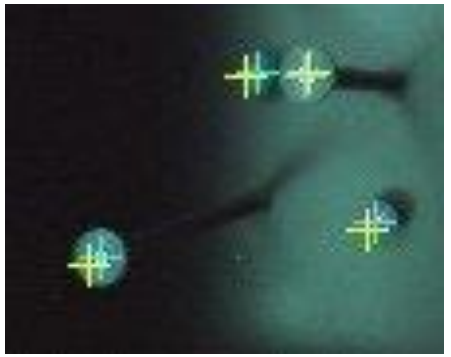
Blue: Detected points
Yellow: Backprojected points

- ***Compute Accuracy Matters!***
- **C++ floats and doubles have the highest accuracy in [-1..1]. Norming the 2D and 3D point pairs by**

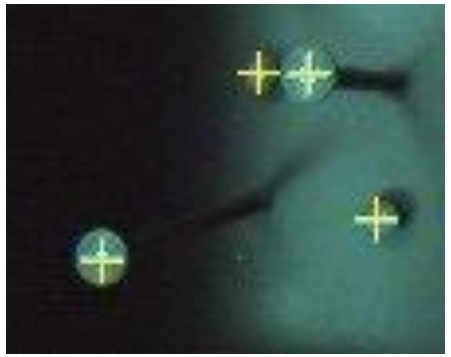
$$\nu = max_i \sqrt{u_i^2 + v_i^2}, (u_i', v_i') = (u_i/\nu, v_i/\nu)$$

$$\mu = max_i \sqrt{x_i^2 + y_i^2 + z_i^2}, (x_i', y_i', z_i') = (x_i/\mu, y_i/\mu, z_i/\mu)$$

then performing the calibration with the normed point pairs and inflating again afterwards by multiplying with ν, μ increases accuracy tremendously.



Direct



Normalized

Backprojection results of calibration with direct and normalized point pairs

- **A couple of notes on this method for camera calibration:**
- **This gives us the projection matrix $\mathrm{P} = (\mathrm{KR} \mid \mathrm{Kt})$, not the calibration matrix and the transformation separately.**
- **We know that KR is the product of an upper right diagonal matrix and an orthogonal matrix.**
- **The *QR Decomposition* of a square matrix A is the decomposition $\mathrm{A} = \mathrm{BC}$, where B is an orthogonal matrix and C an upper right diagonal matrix.**
- **That is the wrong way around but we can QR decompose $\mathrm{A}^\top = (\mathrm{BC})^\top = \mathrm{C}^\top \mathrm{B}^\top$**

  with $\mathrm{C}^\top = \hat{\mathrm{K}}, \mathrm{B}^\top = \hat{\mathrm{R}}$ (the hat denotes an estimate).
- **Various ways of computing a QR decomposition exist (Givens rotations, Gram-Schmidt process)… please look them up if you‘re interested.**

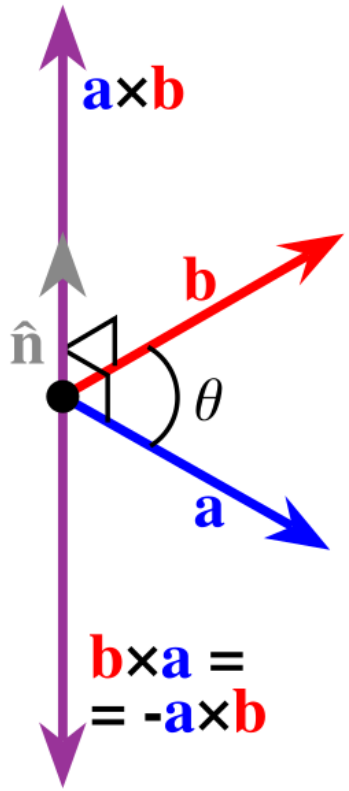
- **The resulting matrix $\mathbf{R}$ is orthogonal (the QR decomposition guarantees that), but its column vectors may not have length 1.As a rotation matrix, it needs to.**

- **With** $\hat{\mathbf{R}} = \begin{pmatrix} \hat{r}_{11} & \hat{r}_{12} & \hat{r}_{13} \\ \hat{r}_{21} & \hat{r}_{22} & \hat{r}_{23} \\ \hat{r}_{31} & \hat{r}_{32} & \hat{r}_{33} \end{pmatrix}, = \begin{pmatrix} \hat{\mathbf{r}}_1 & \hat{\mathbf{r}}_2 & \hat{\mathbf{r}}_3 \end{pmatrix}$

  enforce length one on the first two column vectors $\mathbf{r}_1 = \hat{\mathbf{r}}_1/|\hat{\mathbf{r}}_1|, \mathbf{r}_2 = \hat{\mathbf{r}}_2/|\hat{\mathbf{r}}_2|$

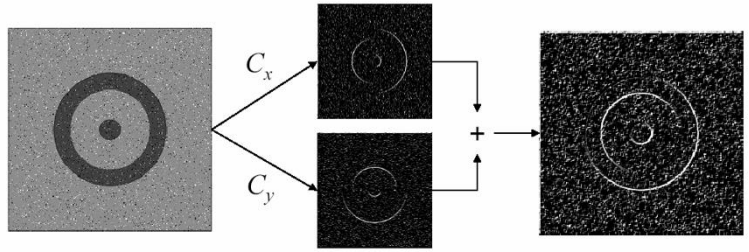
  and recreate the third with the cross product $\mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2 = \begin{pmatrix} r_{12}r_{23} - r_{13}r_{22} \\ r_{13}r_{21} - r_{11}r_{23} \\ r_{11}r_{22} - r_{12}r_{21} \end{pmatrix}$

  for $\mathbf{R} = \begin{pmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{pmatrix}$.



- **Interesting property of orthogonal matrices:** $\mathbf{R}^{-1} = \mathbf{R}^{\top}$
- **We don‘t even have to bother with inversion to solve** $\mathbf{KR}$ **for** $\mathbf{K}$ ☺
- **This finally gives us the solution for the camera intrinsics.**
- **We will skip lens distortion.**

Object Detection from Cameras



- **Cameras lose 3D information through projection. Object detection on cameras operates on contrast and brightness changes.**
- **Trying to teach a system how objects look in a picture therefore often requires the amplification of local contrast and brightness changes.**
- **This principle is valid for most computer vision algorithms:**
  - Simple edge detection
  - Salient feature detection for 3D reconstruction and tracking
  - Early ML algorithms like Viola-Jones wavelet transformation
  - State-of-the-art Convolutional Neural Networks like YOLO.
- **Color information is often less relevant. We will examine grayscale images first.**

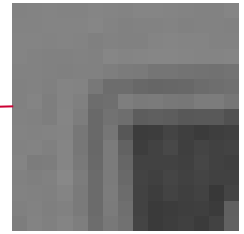
16.01.2026

- **We can think of an image as a 2D function, $\mathrm{Img}(u, v) \rightarrow [0, I_{max}]$, where u, v are the image coordinates and Imax is the maximum intensity allowed by the image.**
- **Object characteristics take the form of local image changes in one or two directions.**
- **We can amplify these changes for later processing by deriving the image in both directions.**
- **The derivative again takes the form of an image.**





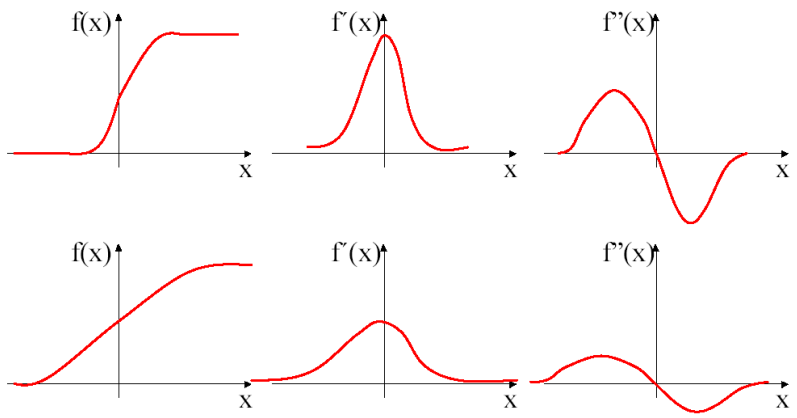
Low change



High change in two directions



High change in one direction



- **Numerical derivative of 2D function delivers another 2D function consisting of**

$$\frac{\partial \mathrm{Img}(u,v)}{\partial u} = \Delta_u \mathrm{Img}(u,v) = \frac{\mathrm{Img}(u,v) - \mathrm{Img}(u-\Delta u, v)}{\Delta u}$$
$$\frac{\partial \mathrm{Img}(u,v)}{\partial v} = \Delta_v \mathrm{Img}(u,v) = \frac{\mathrm{Img}(u,v) - \mathrm{Img}(u, v-\Delta v)}{\Delta u}$$

- **For** $\Delta x = \Delta y = 1$ **, this becomes simply**

$$\Delta_u \mathrm{Img}(u,v) = \mathrm{Img}(u,v) - \mathrm{Img}(u-\Delta u, v)$$
$$\Delta_v \mathrm{Img}(u,v) = \mathrm{Img}(u,v) - \mathrm{Img}(u, v-\Delta v)$$

- **We can write this as a „Filter Mask“ or „Convolution Kernel“:** $C_u = \begin{vmatrix} -1 & 1 \end{vmatrix}, C_v = \begin{vmatrix} -1 \\ 1 \end{vmatrix}$

- **Convolution kernels follow the general expression**

$$g(u,v) = C * \mathrm{Img}(u,v) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} c_{i,j} \mathrm{Img}(u-i, v-j)$$

where $g(u,v)$ is the filtered image and C is the filter kernel of size $2a+1, 2b+1$ (adapt the formula for even-sized filter masks).

- **Note that u and v are bounded – the image starts at (0,0) and ends at $u_{\max}, v_{\max}$. What happens when the convolution operation accesses image values beyond those bounds is defined by the implementation.**
- **Convolution kernels take many different forms. If the sum of all elements is not 1, the resulting image will be brighter or darker than the original image.**

$$\begin{vmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{vmatrix}$$

Identity

$$\frac{1}{16} \begin{vmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{vmatrix}$$

3x3 Gaussian blur approx.

$$\begin{vmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{vmatrix}$$

3x3 Gaussian blur approx.

- **The most interesting computer vision application for convolution is edge detection or partial derivation.**
- **If edge direction is important, the image can be convoluted with the Sobel operators (Irwin Sobel, 1968):**

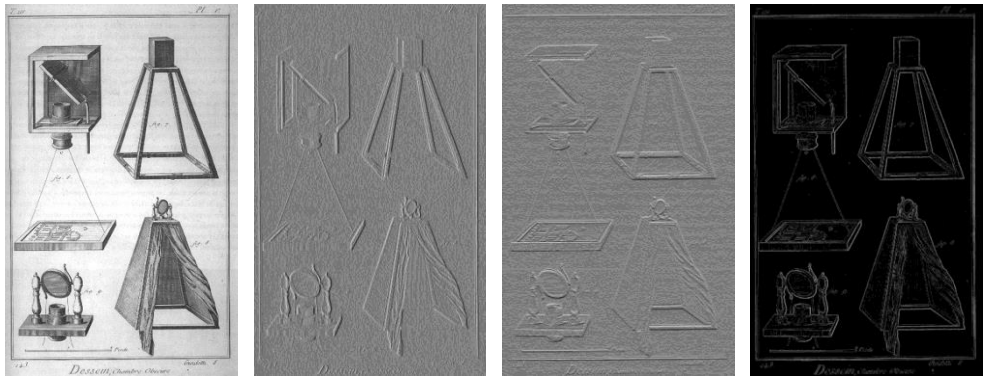
$$C_{\mathrm{Sx}} = \begin{vmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{vmatrix}, C_{\mathrm{Sy}} = \begin{vmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{vmatrix}$$

- **The gradient image can then be obtained by**

$$g(u,v) = \sqrt{(C_{\mathrm{Sx}} * \mathrm{Img}(u,v))^2 + (C_{\mathrm{Sy}} * \mathrm{Img}(u,v))^2}$$

and an image representing the edge direction by

$$\Theta(u,v) = tan^{-1}((C_{\mathrm{Sx}} * \mathrm{Img}(u,v))/(C_{\mathrm{Sy}} * \mathrm{Img}(u,v)))$$



Original, Sobel in u, Sobel in v, absolute combination
Source: Wikipedia



Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!